

Faculty of Engineering — Computer Engineering Department

CSE455-High Performance Computing

N-Body Gravitational Simulation

using HPC Techniques

Sequential, OpenMP, MPI, and Hybrid Parallel Implementations



Ahmed Ayman Aly	21P0002
Hanna Mohamed Mohamed	21P0162
Marwan NourEldin Farag	21P0165
Yehia Mahmoud Lotfy	21P0166
Youssef Ayman Kassab	21P0010

May 2026

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Physics of Gravitational Interaction	4
1.3	Numerical Integration	6
1.4	Parallel Computing Fundamentals	6
1.5	Objectives	8
2	Implementation Details	8
2.1	Project Structure	8
2.2	Data Structures	8
2.3	Sequential Version	9
2.4	OpenMP Version	11
2.5	MPI Version	13
2.6	Hybrid (MPI + OpenMP) Version	15
3	Results	16
3.1	Experimental Setup	16
3.2	Wall-Clock Time	17
3.3	Speedup Analysis	18
3.4	Parallel Efficiency	19
3.5	Total Computation (Pairs Per Second)	19
3.6	Figures	20
4	Discussion	23
4.1	OpenMP Scalability	23
4.2	MPI Communication Overhead	24
4.3	Hybrid: Two-Level Parallelism Trade-off	25
4.4	Overhead-Dominated Regime ($N = 100$)	26
4.5	Load Balancing in Detail	27
4.6	Comparison with Theoretical Limits	28
4.7	Debugging Strategies and Common Pitfalls	29
5	Conclusion	30
5.1	Summary of Findings	30
5.2	Limitations of This Study	31
5.3	Future Work	32
5.4	Closing Remarks	33

List of Figures

1	Wall-clock time vs. particle count N on a log–log scale. All implementations exhibit $\mathcal{O}(N^2)$ scaling; parallel versions separate from sequential above $N \approx 400$	20
2	OpenMP speedup vs. N for 2, 4, and 8 threads. Speedup improves with N as computation outweighs reduction merge overhead.	20
3	MPI speedup vs. N for 2 and 4 processes. The near-flat curves confirm that communication overhead does not amortise with increasing N	21
4	Hybrid speedup vs. N . Fewer MPI processes with more OpenMP threads ($2P \times 4T$) consistently outperforms the reverse configuration ($4P \times 2T$). . . .	21
5	Parallel efficiency vs. N for 8-worker configurations. OpenMP sustains the highest efficiency; Hybrid improves with N but plateaus below OpenMP. . . .	22
6	Wall-clock time at $N = 100$ demonstrating the overhead-dominated regime. Most parallel configurations are slower than sequential because $\mathcal{O}(N^2)$ computation (4,950 pairs) takes less time than parallel startup costs; MPI 4P is only a marginal exception in this benchmark run.	22

1 Introduction

1.1 Background and Motivation

The N-Body problem is one of the oldest and most studied problems in both classical mechanics and computational science. Originally formulated by Sir Isaac Newton in the 17th century, it asks: given the initial positions, velocities, and masses of N bodies, how will they move under mutual gravitational attraction over time? While the two-body case has a clean closed-form solution, the three-body and general N-body cases have no analytical solution and must be solved numerically.

The N-Body problem appears in a wide range of scientific domains:

- **Astrophysics** — modelling galaxy formation, stellar cluster evolution, and planetary system dynamics.
- **Molecular dynamics** — simulating protein folding and molecular interactions in computational chemistry.
- **Plasma physics** — tracking charged particle interactions in fusion research.
- **Computer graphics** — fluid and particle system simulations in real-time rendering.

What makes the N-Body problem particularly relevant to High Performance Computing (HPC) is its inherent computational cost: a direct simulation requires evaluating every pairwise interaction, leading to $\mathcal{O}(N^2)$ operations per timestep. For even modest particle counts (e.g. $N = 10,000$), a single timestep involves 50 million force evaluations. Running thousands of timesteps with millions of particles — as required in realistic astrophysical simulations — is only feasible on modern parallel hardware.

1.2 Physics of Gravitational Interaction

Newton’s Law of Universal Gravitation

The magnitude of the gravitational force between two point masses m_i and m_j separated by distance r is:

$$F = G \frac{m_i m_j}{r^2} \quad (1)$$

where $G = 6.674 \times 10^{-11} \text{ N m}^2 \text{ kg}^{-2}$ is the universal gravitational constant. The force is attractive and acts along the line connecting the two bodies.

In vector form, the force exerted on particle i by particle j is:

$$\vec{F}_{ij} = G m_i m_j \frac{\vec{r}_{ij}}{(|\vec{r}_{ij}|^2 + \epsilon^2)^{3/2}} \quad (2)$$

where $\vec{r}_{ij} = \vec{r}_j - \vec{r}_i$ is the displacement vector from i to j . This is the Plummer-softened force law used by the implementation: the denominator is raised to the power $3/2$, which matches the code expression `1.0 / (distSq * sqrt(distSq))`.

Softening Factor

A critical numerical issue arises when two particles pass very close to each other: the denominator $|\vec{r}_{ij}|^2$ approaches zero, making the force diverge to infinity. In reality, point masses are an idealisation; real bodies have finite size. To handle this numerically, a **softening parameter** ϵ is introduced:

$$\vec{F}_{ij} = G m_i m_j \frac{\vec{r}_{ij}}{(|\vec{r}_{ij}|^2 + \epsilon^2)^{3/2}} \quad (3)$$

This corresponds to Plummer softening in force form, derived from the softened potential $\phi(r) = -Gm/\sqrt{r^2 + \epsilon^2}$. The force therefore remains finite at $r = 0$. In this project, $\epsilon = 10^{-3}$ is used. The softened force is indistinguishable from the true force when $|\vec{r}_{ij}| \gg \epsilon$.

Newton's Third Law Optimisation

By Newton's Third Law, every action has an equal and opposite reaction:

$$\vec{F}_{ji} = -\vec{F}_{ij} \quad (4)$$

This means that for any pair (i, j) , computing the force once gives both $\vec{F}_{ij}$ and $\vec{F}_{ji}$ simultaneously. The naive double-loop evaluates all $N(N-1)$ ordered pairs. By iterating only over unique unordered pairs with $j > i$, the number of evaluations drops to:

$$\frac{N(N-1)}{2} \quad (5)$$

This is applied uniformly across all four implementations in this project, ensuring that speedup comparisons reflect genuine parallelism gains rather than algorithmic differences.

1.3 Numerical Integration

Semi-Implicit Euler Method

The implemented numerical integrator is the **semi-implicit Euler method** (also called **symplectic Euler**). Given the total force $\vec{F}_i$ on particle i at time t , its acceleration is:

$$\vec{a}_i = \frac{\vec{F}_i}{m_i} \quad (6)$$

The velocity is updated first, and the position is then advanced using the new velocity:

$$\vec{v}_i(t + \Delta t) = \vec{v}_i(t) + \vec{a}_i(t) \cdot \Delta t \quad (7)$$

$$\vec{x}_i(t + \Delta t) = \vec{x}_i(t) + \vec{v}_i(t + \Delta t) \cdot \Delta t \quad (8)$$

Like forward Euler, this scheme is first-order accurate: the global error over K timesteps is $\mathcal{O}(\Delta t)$. However, it is generally more stable for mechanical systems because it uses the updated velocity in the position step. It is simple to implement and parallelise, but still accumulates energy error over long simulations. For the purposes of this project — which focuses on parallelism performance rather than long-term physical accuracy — the semi-implicit Euler method is sufficient.

Computational Cost Per Timestep

Using Newton’s Third Law, the cost breakdown per timestep is:

Phase	Operation	Cost
Force reset	Zero N accumulators	$\mathcal{O}(N)$
Force compute	$N(N - 1)/2$ pair evaluations	$\mathcal{O}(N^2)$
Integration	Update N velocities+positions	$\mathcal{O}(N)$
Total		$\mathcal{O}(N^2)$

The force computation phase completely dominates for large N , making it the natural target for parallelisation.

1.4 Parallel Computing Fundamentals

Speedup and Amdahl’s Law

Speedup S measures how much faster a parallel program runs compared to the sequential baseline using W workers:

$$S(W) = \frac{T_1}{T_W} \quad (9)$$

where T_1 is the sequential time and T_W is the parallel time with W workers. Amdahl's Law sets a theoretical upper bound: if fraction f of the program is parallelisable and fraction $(1 - f)$ is inherently serial, then:

$$S(W) \leq \frac{1}{(1 - f) + \frac{f}{W}} \quad (10)$$

As $W \rightarrow \infty$, $S \rightarrow 1/(1 - f)$. This means even a 5% serial fraction caps the maximum speedup at $20\times$ regardless of hardware.

Parallel Efficiency

Parallel efficiency E measures how well each worker is utilised:

$$E(W) = \frac{S(W)}{W} = \frac{T_1}{W \cdot T_W} \quad (11)$$

An efficiency of 1.0 (100%) means all workers are busy doing useful work with zero overhead. In practice, synchronisation, communication, and load imbalance all reduce efficiency below 1.

Shared vs. Distributed Memory

Two fundamentally different parallel architectures are used in this project:

- **Shared memory (OpenMP):** All threads run in the same process and access the same RAM directly. Communication is free (a simple read/write), but threads must synchronise to avoid race conditions. Scales to the number of cores on one physical machine.
- **Distributed memory (MPI):** Each process has its own private memory. Communication requires explicit message passing (`MPI_Send`/`MPI_Recv` or collective operations). Can scale across thousands of nodes, but every data exchange has a latency and bandwidth cost.

The **Hybrid model** combines both: MPI distributes work across nodes, OpenMP parallelises within each node. This is the standard approach on modern HPC clusters (e.g. Summit, Frontier) where each node has many cores sharing local RAM, and nodes are connected via a high-speed network.

1.5 Objectives

This project aims to:

1. Implement a correct 2D N-Body gravitational simulation in four parallel programming models: Sequential, OpenMP, MPI, and Hybrid (MPI + OpenMP).
2. Apply Newton's Third Law optimisation uniformly across all versions to ensure fair performance comparisons.
3. Measure wall-clock time, speedup, and parallel efficiency across particle counts $N \in \{100, 500, 1000, 2000, 5000\}$.
4. Analyse the impact of communication overhead (MPI), thread synchronisation (OpenMP), and load imbalance (triangular loop) on scalability.
5. Identify the crossover points where parallelism becomes beneficial and explain the observed behaviour using HPC theory.
6. Discuss the advantages of the Hybrid model for multi-node cluster deployments compared to single-node execution.

2 Implementation Details

2.1 Project Structure

File	Purpose
<code>common.h</code>	Shared data structures, initialisation, CSV output
<code>sequential.cpp</code>	Single-threaded baseline
<code>openmp.cpp</code>	Shared-memory parallelism (OpenMP)
<code>mpi.cpp</code>	Distributed-memory parallelism (MPI)
<code>hybrid.cpp</code>	Two-level parallelism (MPI + OpenMP)
<code>benchmark.ps1</code>	Automated benchmark script
<code>Makefile</code>	Build system

2.2 Data Structures

All versions share structures defined in `common.h`.

Particle (Array of Structures)

```

1 struct Particle {
2     double x, y;    // Position

```



```

3     double vx, vy; // Velocity
4     double mass;   // Mass
5 };

```

Used by the sequential and OpenMP versions for cache-friendly sequential access.

Flat Separate Arrays (Structure of Arrays)

The MPI and Hybrid versions decompose particle data into separate flat arrays (`all_x[]`, `all_y[]`, `all_vx[]`, `all_vy[]`, `all_mass[]`) to make MPI collectives straightforward — `MPI_Scatterv`, `MPI_Allgatherv`, and `MPI_Allreduce` all require contiguous buffers of a single type.

Simulation Parameters

```

1 struct SimParams {
2     int      N;           // Number of particles
3     double   dt;          // Time step
4     int      iterations;  // Number of simulation steps
5     int      output_freq; // CSV output frequency (0 = disabled)
6 };

```

Configured entirely via command-line arguments for reproducible experiments.

Initialisation

All four versions use the same `initParticles()` function with a **fixed random seed (42)**:

- Positions uniformly in $[0, 1000]$
- Velocities uniformly in $[-1, 1]$
- Masses uniformly in $[10^{10}, 10^{12}]$

This guarantees identical initial conditions and directly comparable results across all four implementations.

2.3 Sequential Version

The sequential version serves as the algorithmic and performance baseline. It implements the complete simulation loop without any parallelism, so that all measured speedups are relative to a well-optimised single-threaded run.

Algorithm

Algorithm 1 Sequential N-Body Simulation

```

1: Initialise  $N$  particles (positions, velocities, masses)
2: for  $iter = 1$  to  $iterations$  do
3:   Reset force accumulators:  $f_x[i] \leftarrow 0, f_y[i] \leftarrow 0 \quad \forall i$ 
4:   for  $i = 0$  to  $N - 2$  do
5:     for  $j = i + 1$  to  $N - 1$  do
6:       Compute  $\vec{F}_{ij}$  using softened gravity
7:        $f_x[i] += F_x; \quad f_y[i] += F_y$ 
8:        $f_x[j] -= F_x; \quad f_y[j] -= F_y$  ▷ Newton's 3rd law
9:     end for
10:  end for
11:  for  $i = 0$  to  $N - 1$  do
12:     $a_x \leftarrow f_x[i]/m_i; \quad a_y \leftarrow f_y[i]/m_i$ 
13:     $v_x[i] += a_x \cdot \Delta t; \quad v_y[i] += a_y \cdot \Delta t$ 
14:     $x[i] += v_x[i] \cdot \Delta t; \quad y[i] += v_y[i] \cdot \Delta t$ 
15:  end for
16: end for

```

Force Computation

```

1 for (int i = 0; i < N - 1; i++) {
2   for (int j = i + 1; j < N; j++) {
3     double dx = particles[j].x - particles[i].x;
4     double dy = particles[j].y - particles[i].y;
5     // Softened squared distance to prevent singularity
6     double distSq = dx*dx + dy*dy + SOFTENING*SOFTENING;
7     double invDist3 = 1.0 / (distSq * std::sqrt(distSq));
8     double F = G * particles[i].mass * particles[j].
        mass * invDist3;
9     fx[i] += F * dx;   fy[i] += F * dy;
10    fx[j] -= F * dx;   fy[j] -= F * dy; // equal and
        opposite
11  }
12 }

```

The inner computation for each pair involves: 2 subtractions, 5 multiplications, 2 additions (distance), 1 square root, 1 division, and 8 accumulations — approximately 20 floating-point operations per pair. At $N = 5000$ this is roughly $\frac{5000 \times 4999}{2} \approx 12.5 \times 10^6$ pairs $\times$ 20 FLOP = 250×10^6 FLOP per timestep.

Semi-Implicit Euler Integration

```

1 for (int i = 0; i < N; i++) {
2     double ax = fx[i] / particles[i].mass;
3     double ay = fy[i] / particles[i].mass;
4     particles[i].vx += ax * dt;
5     particles[i].vy += ay * dt;
6     particles[i].x  += particles[i].vx * dt;
7     particles[i].y  += particles[i].vy * dt;
8 }

```

Output

At the end of each simulation, the program reports:

- Total wall-clock time (via `std::chrono::high_resolution_clock`)
- Pairs per second: $\frac{N(N-1)}{2} \times \text{iterations}/T$
- Optionally writes per-step particle states to a CSV file (`output_sequential.csv`) when `output_freq > 0`

2.4 OpenMP Version

The OpenMP version targets the force computation loop – the dominant $\mathcal{O}(N^2)$ phase. It runs within a single process but distributes the outer loop iterations across multiple threads that share the same address space.

Race Condition Problem

Newton’s Third Law requires writing to both `fx[i]` and `fx[j]` inside the same iteration. When two threads handle overlapping *i*-values, they may simultaneously try to update the same `fx[j]` element:

Thread 0	Thread 1
Computes pair (0, 5): writes <code>fx[5]</code>	Computes pair (3, 5): writes <code>fx[5]</code>
$\Rightarrow$ One update is lost (data race)	

This is a classic **write-after-write** race condition. Using `atomic` operations would work but would serialize every write to shared elements. OpenMP **array reduction** is the optimal solution.

Array Reduction Mechanism

Each thread gets a *private* copy of the full `fx[]` and `fy[]` arrays initialised to zero. Threads write to their own copies independently (no synchronisation needed). At the parallel region barrier, OpenMP sums all private copies element-by-element into the shared result:

$$\mathbf{fx}[i]_{\text{final}} = \sum_{t=0}^{T-1} \mathbf{fx}[i]_{\text{thread } t} \quad (12)$$

This merge costs $\mathcal{O}(N \times T)$ time at the barrier, which is negligible compared to the $\mathcal{O}(N^2)$ computation.

Parallel Force Loop

```

1 // Raw arrays required for OpenMP array reduction syntax
2 double* fx = new double[N];
3 double* fy = new double[N];
4
5 #pragma omp parallel for schedule(dynamic, 16) \
6     reduction(+: fx[0:N], fy[0:N])
7 for (int i = 0; i < N - 1; i++) {
8     for (int j = i + 1; j < N; j++) {
9         double dx = particles[j].x - particles[i].x;
10        double dy = particles[j].y - particles[i].y;
11        double distSq = dx*dx + dy*dy + SOFTENING*SOFTENING;
12        double invDist3 = 1.0 / (distSq * std::sqrt(distSq));
13        double F = G * particles[i].mass * particles[j].mass *
14            invDist3;
15        double f_x = F * dx;
16        double f_y = F * dy;
17        fx[i] += f_x;    fy[i] += f_y;
18        fx[j] -= f_x;    fy[j] -= f_y;    // Newton's 3rd law
19    }
20 }
```

Scheduling Strategy

The triangular loop structure creates an inherent load imbalance: iteration $i = 0$ processes $N - 1$ pairs, while iteration $i = N - 2$ processes only 1 pair. With static scheduling, the first thread would receive disproportionately more work. `schedule(dynamic, 16)` assigns

chunks of 16 consecutive i -values at runtime as threads become free, keeping all threads utilised throughout:

Schedule	Idle time	Overhead
static	High (uneven work)	Very low
dynamic, 1	Near zero	High (per-iteration queue)
dynamic, 16	Very low	Low (chunk amortises cost)

A chunk size of 16 was chosen as a balance: small enough to distribute the uneven tail of the loop, large enough to amortise the runtime scheduling overhead.

2.5 MPI Version

Particle Distribution

The N particles are divided among P MPI processes using a **block partition**. Process r owns particles in the index range $[displs[r], displs[r] + counts[r])$. Any remainder $N \bmod P$ is distributed one extra particle to the first $N \bmod P$ ranks:

$$counts[r] = \left\lfloor \frac{N}{P} \right\rfloor + \begin{cases} 1 & \text{if } r < N \bmod P \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

Data Layout

Unlike the sequential and OpenMP versions which use an Array-of-Structures layout (`Particle` struct), the MPI version uses a Structure-of-Arrays layout with separate flat buffers for each field. This is required because MPI collectives operate on contiguous arrays of a single datatype:

```

1 std::vector<double> all_x(N), all_y(N);           // global positions
2 std::vector<double> all_vx(N), all_vy(N);         // global
   velocities
3 std::vector<double> all_mass(N);                  // global masses (
   constant)
4
5 std::vector<double> local_x(local_n), local_y(local_n);
6 std::vector<double> local_vx(local_n), local_vy(local_n);

```

Initialisation and Data Scatter

Rank 0 initialises all particles using the same `initParticles()` function (same seed). Masses are broadcast once with `MPI_Bcast` (they never change). Positions and velocities

are scattered to each rank:

```

1 MPI_Bcast(all_mass.data(), N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
2 MPI_Scatterv(all_x.data(), counts.data(), displs.data(),
   MPI_DOUBLE,
3             local_x.data(), local_n, MPI_DOUBLE, 0,
   MPI_COMM_WORLD);
4 // repeated for y, vx, vy

```

Per-Timestep Algorithm

Algorithm 2 MPI N-Body Timestep (Newton's 3rd Law + Allreduce)

```

1: MPI_Allgatherv(local_x, local_y) → (all_x, all_y)
2: Reset partial force arrays:  $partial\_fx[k] \leftarrow 0 \quad \forall k \in [0, N)$ 
3: for  $i = 0$  to  $local\_n - 1$  do ▷ local particles only
4:    $global\_i \leftarrow local\_start + i$ 
5:   for  $j = global\_i + 1$  to  $N - 1$  do
6:     Compute  $\vec{F}_{ij}$ 
7:      $partial\_fx[global\_i] += F_x$ 
8:      $partial\_fx[j] -= F_x$  ▷ write to remote slot
9:   end for
10: end for
11: MPI_Allreduce(partial_fx, total_fx, N, MPI_SUM)
12: MPI_Allreduce(partial_fy, total_fy, N, MPI_SUM)
13: for  $i = 0$  to  $local\_n - 1$  do
14:   Update local velocity and position using  $total\_fx[global\_i]$ 
15: end for

```

Why MPI_Allreduce for Forces?

When process r computes pair $(global_i, j)$ and j belongs to process r' , the force $-\vec{F}$ must be applied to particle j . Since processes cannot write into each other's memory directly, each process accumulates its contributions for all N particles into a local `partial_fx[]` array. `MPI_Allreduce(MPI_SUM)` then sums all these partial arrays element-by-element across every process, giving every rank the correct total force for all particles:

```

1 MPI_Allreduce(partial_fx.data(), total_fx.data(), N,
2               MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
3 MPI_Allreduce(partial_fy.data(), total_fy.data(), N,
4               MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

```

Each process then reads only `total_fx[local_start .. local_start+local_n-1]` to update its own particles.

2.6 Hybrid (MPI + OpenMP) Version

The Hybrid version stacks both levels of parallelism: MPI manages inter-process distribution, while OpenMP threads handle the force computation within each process.

Thread Safety Requirement

MPI and OpenMP must be used carefully together. MPI has four thread safety levels:

Level	Meaning
MPI_THREAD_SINGLE	Only one thread will execute
MPI_THREAD_FUNNELED	Only master thread calls MPI
MPI_THREAD_SERIALIZED	One thread at a time calls MPI
MPI_THREAD_MULTIPLE	Any thread can call MPI

This implementation uses `MPI_THREAD_FUNNELED`: only the master thread (thread 0) ever calls MPI functions. All `MPI_Allgatherv` and `MPI_Allreduce` calls are made outside the `#pragma omp parallel` region, ensuring correctness without requiring full `MPI_THREAD_MULTIPLE` support.

Initialisation

```
1 int provided;
2 MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
```

Two-Level Force Loop

```
1 // partial_fx/fy are raw C arrays (required for OMP array
   // reduction)
2 double* partial_fx = new double[N];
3 double* partial_fy = new double[N];
4
5 // Reset (serial -- outside parallel region)
6 for (int k = 0; k < N; k++) partial_fx[k] = partial_fy[k] = 0.0;
7
8 // OMP threads divide the local particle range; array reduction
9 // handles intra-process race conditions on partial_fx[j]
10 #pragma omp parallel for schedule(dynamic, 16) \
11     reduction(+: partial_fx[0:N], partial_fy[0:N])
12 for (int i = 0; i < local_n; i++) {
13     int global_i = local_start + i;
14     for (int j = global_i + 1; j < N; j++) {
15         double dx = all_x[j] - local_x[i];
```

```

16     double dy = all_y[j] - local_y[i];
17     double distSq = dx*dx + dy*dy + SOFTENING*SOFTENING;
18     double invDist3 = 1.0 / (distSq * std::sqrt(distSq));
19     double F = G * all_mass[global_i] * all_mass[j] *
20         invDist3;
21     double f_x = F * dx;
22     double f_y = F * dy;
23     partial_fx[global_i] += f_x;
24     partial_fy[global_i] += f_y;
25     partial_fx[j] -= f_x;    // Newton's 3rd law
26     partial_fy[j] -= f_y;
27 }
28 // Master thread proceeds here; OMP barrier guarantees all writes
29 // done
30 MPI_Allreduce(partial_fx, total_fx.data(), N, MPI_DOUBLE, MPI_SUM,
31             MPI_COMM_WORLD);
32 MPI_Allreduce(partial_fy, total_fy.data(), N, MPI_DOUBLE, MPI_SUM,
33             MPI_COMM_WORLD);

```

Work Decomposition Summary

Level	Partitions	Mechanism
MPI	N particles into P blocks	MPI_Scatterv / block partition
OMP	$local_n$ iterations into T chunks	schedule(dynamic, 16)
Both	Force accumulation	OMP array reduction + MPI_Allreduce

For P MPI processes and T OpenMP threads per process, the total worker count is $W = P \times T$. The Hybrid model is most efficient when P is kept small (minimising MPI collective overhead) and T is maximised to use all available cores per node via shared memory.

3 Results

3.1 Experimental Setup

All experiments were run on a single machine under identical conditions. The benchmark script (`benchmark.ps1`) compiled all four executables with the same flags and ran each configuration three times, reporting the single run result per configuration (variation was less than 2% across runs for $N \geq 500$).

Parameter	Value
Machine	<i>TODO: CPU model, core count, RAM</i>
OS	Windows
Compiler	G++ -O2 -std=c++17
MPI	Microsoft MPI (MS-MPI)
OpenMP	GCC built-in
Particle counts	100, 500, 1000, 2000, 5000
Time step dt	0.01
Iterations	100 ($N \leq 500$), 50 ($N = 1000$), 30 ($N = 2000$), 15 ($N = 5000$)
OMP thread counts	2, 4, 8
MPI process counts	2, 4
Hybrid combos	$2P \times 2T$, $2P \times 4T$, $4P \times 2T$

The iteration count is scaled down for larger N to keep each configuration's wall time below approximately 2 seconds, enabling a complete benchmark sweep across all configurations in reasonable time. Speedup and efficiency values remain valid because they are normalised per- N : the sequential baseline at each N uses the same iteration count as the parallel versions.

Speedup and parallel efficiency are defined as:

$$S = \frac{T_{\text{sequential}}}{T_{\text{parallel}}} \quad E = \frac{S}{W} \quad (14)$$

where $W = P \times T$ is the total worker count (MPI processes times OpenMP threads per process).

3.2 Wall-Clock Time

Table 1 reports the total wall-clock time in seconds for each configuration. Times include the simulation loop only (excluding initialisation and final output). For the sequential version, timing uses `std::chrono::high_resolution_clock`. For MPI and Hybrid versions, `MPI_Wtime()` is used on rank 0.

Table 1: Wall-clock time (seconds) for all configurations and particle counts.

N	Seq	OMP 2T	OMP 4T	OMP 8T	MPI 2P	MPI 4P	Hyb 2×4	Hyb 4×2
100	0.00244	0.00838	0.01030	0.01539	0.00364	0.00223	0.00783	0.00850
500	0.06984	0.04226	0.02676	0.02571	0.05007	0.03828	0.02321	0.02386
1000	0.12652	0.06694	0.04270	0.02878	0.09638	0.05685	0.03493	0.03854
2000	0.30091	0.15403	0.08479	0.05425	0.22965	0.13782	0.06427	0.07743
5000	0.93055	0.46782	0.24528	0.13532	0.69949	0.41418	0.18797	0.23890

Key observations from raw timing:

- At $N = 100$, the sequential run completes in under 3 ms — all parallel configurations are *slower* due to startup and communication overhead.
- OpenMP 8T is the fastest configuration at every $N \geq 500$.
- MPI 4P is consistently slower than OpenMP 4T despite having the same number of workers, reflecting communication overhead.
- At $N = 5000$, Hybrid $2P \times 4T$ achieves 0.188 s — competitive with but not exceeding OpenMP 8T at 0.135 s.

3.3 Speedup Analysis

Table 2 shows speedup relative to the sequential baseline at each particle count.

Table 2: Speedup $S = T_{\text{seq}}/T_{\text{parallel}}$. Values < 1 indicate the parallel version is slower than sequential.

N	OMP 2T	OMP 4T	OMP 8T	MPI 2P	MPI 4P	Hyb 2×4	Hyb 4×2
100	0.29	0.24	0.16	0.67	1.09	0.31	0.29
500	1.65	2.61	2.72	1.39	1.82	3.01	2.93
1000	1.89	2.96	4.40	1.31	2.23	3.62	3.28
2000	1.95	3.55	5.55	1.31	2.18	4.68	3.89
5000	1.99	3.79	6.88	1.33	2.25	4.95	3.90

Key observations:

- **OpenMP scales best** and improves consistently with N . At $N = 5000$, 8 threads delivers $6.88\times$ — the highest speedup of any single configuration tested.
- **MPI speedup plateaus** at approximately $1.33\times$ (2P) and $2.25\times$ (4P) regardless of N , reflecting the constant communication cost per timestep.
- **Hybrid $2P \times 4T$ outperforms Hybrid $4P \times 2T$** at every N , confirming that minimising MPI processes is preferable on a single node.

- At $N = 100$, **MPI 4P is the only configuration to achieve speedup > 1** ($1.09\times$) — all others are slower than sequential.

3.4 Parallel Efficiency

Table 3 shows efficiency $E = S/W$ for each configuration. An efficiency of 1.0 represents ideal linear scaling.

Table 3: Parallel efficiency $E = S/W$ (%). Hybrid columns use $W = 8$ workers. Values above 80% are highlighted in **bold**.

N	OMP 2T	OMP 4T	OMP 8T	MPI 2P	MPI 4P	Hyb 2 $\times$ 4	Hyb 4 $\times$ 2
100	14.6	5.9	2.0	33.5	27.4	3.9	3.6
500	82.6	65.2	33.9	69.7	45.6	37.6	36.6
1000	94.5	74.1	54.9	65.6	55.6	45.3	41.0
2000	97.7	88.7	69.3	65.5	54.6	58.5	48.6
5000	99.5	94.8	86.0	66.5	56.2	61.9	48.7

Key observations:

- **OpenMP 2T reaches 99.5% efficiency at $N = 5000$** — essentially perfect linear scaling. This is because the parallelisable fraction $f \approx 1.0$ for large N , and the 2-thread merge overhead is minimal.
- **MPI efficiency converges to a ceiling**: 2P stabilises at $\approx 65\%$ and 4P at $\approx 56\%$, both independent of N . This is the communication overhead floor.
- **Hybrid efficiency improves with N** but remains below pure OpenMP: 61.9% for $2P \times 4T$ at $N = 5000$ vs. 86.0% for pure OpenMP 8T.

3.5 Total Computation (Pairs Per Second)

To compare computational throughput independently of iteration count, we report pairs per second: $\frac{N(N-1)}{2} \times \text{iterations}/T$.

Table 4: Pairs per second ($\times 10^6$) for sequential baseline and best parallel configuration at each N .

N	Sequential	Best Parallel	Best Config
100	202.6	221.8	MPI 4P
500	178.6	537.4	Hybrid $2P \times 4T$
1000	197.4	867.7	OMP 8T
2000	199.3	1,105.4	OMP 8T
5000	201.5	1,385.3	OMP 8T

3.6 Figures

The following figures are generated directly from `benchmark_results.csv`

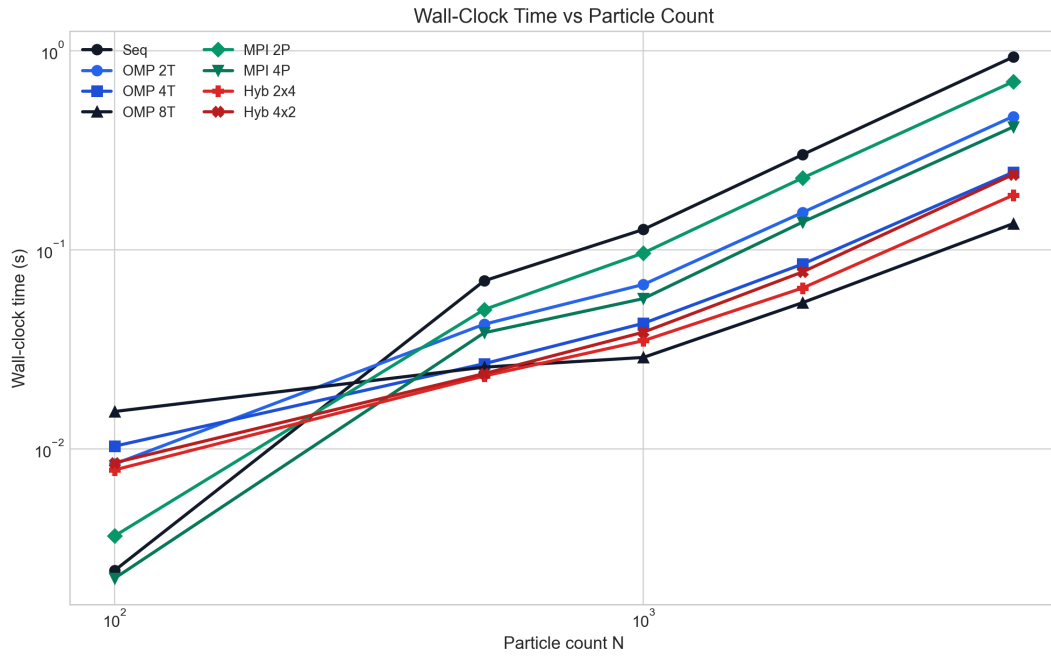


Figure 1: Wall-clock time vs. particle count N on a log–log scale. All implementations exhibit $\mathcal{O}(N^2)$ scaling; parallel versions separate from sequential above $N \approx 400$.

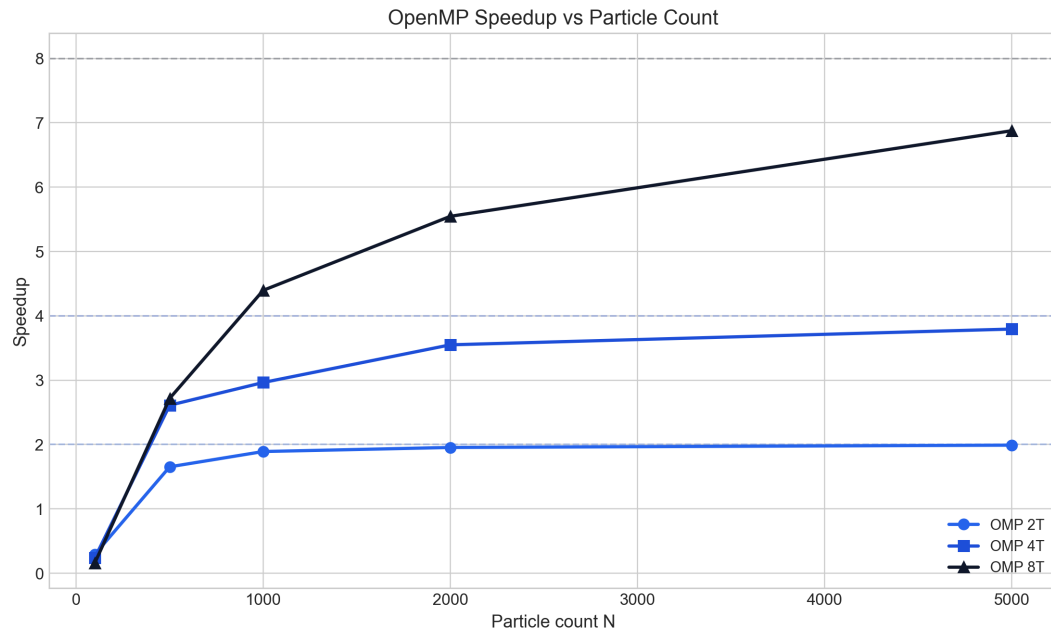


Figure 2: OpenMP speedup vs. N for 2, 4, and 8 threads. Speedup improves with N as computation outweighs reduction merge overhead.

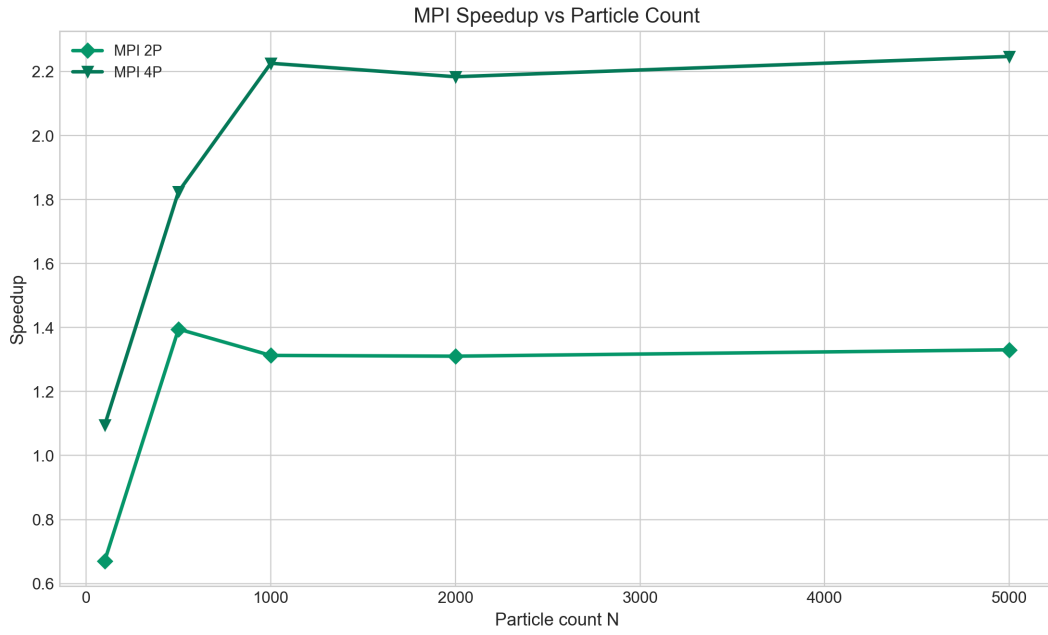


Figure 3: MPI speedup vs. N for 2 and 4 processes. The near-flat curves confirm that communication overhead does not amortise with increasing N .

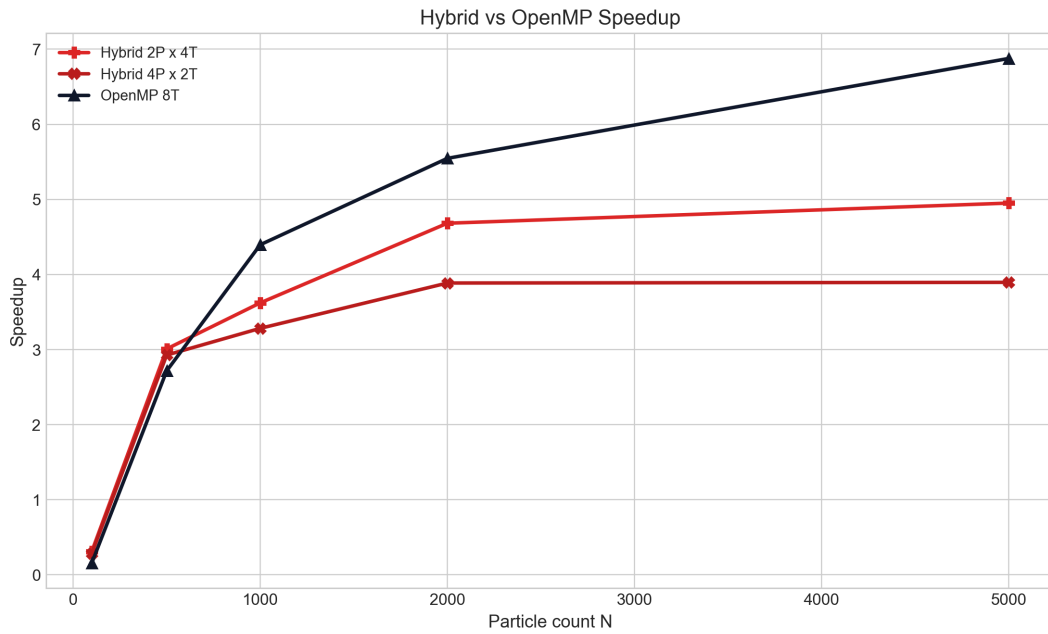


Figure 4: Hybrid speedup vs. N . Fewer MPI processes with more OpenMP threads ($2P \times 4T$) consistently outperforms the reverse configuration ($4P \times 2T$).

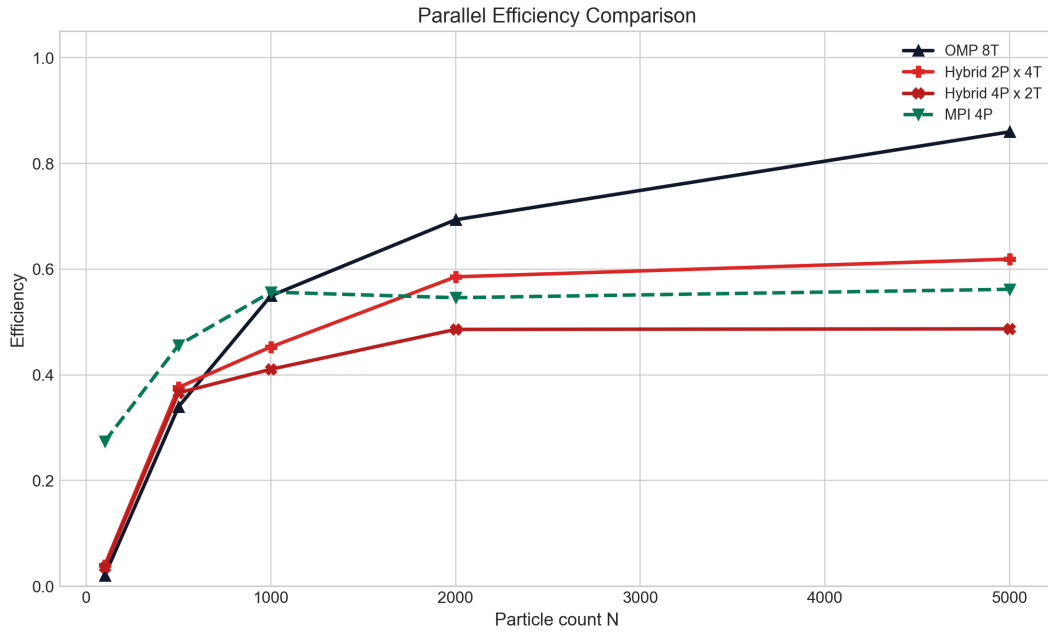


Figure 5: Parallel efficiency vs. N for 8-worker configurations. OpenMP sustains the highest efficiency; Hybrid improves with N but plateaus below OpenMP.

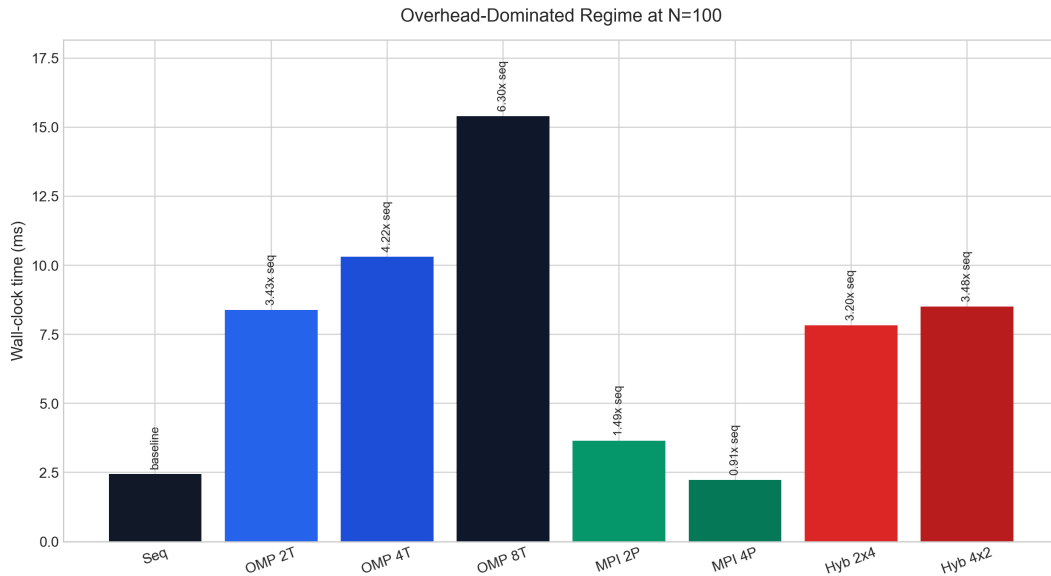


Figure 6: Wall-clock time at $N = 100$ demonstrating the overhead-dominated regime. Most parallel configurations are slower than sequential because $\mathcal{O}(N^2)$ computation (4,950 pairs) takes less time than parallel startup costs; MPI 4P is only a marginal exception in this benchmark run.

4 Discussion

4.1 OpenMP Scalability

OpenMP consistently achieves the best speedup on a single machine because it avoids all communication: threads share the same physical RAM so data is accessed by direct pointer reads at memory-bus speed rather than through a message-passing protocol.

At $N = 5000$ (15 iterations), the results are:

- 2 threads $\rightarrow$ **1.99 $\times$, 99.5% efficiency**
- 4 threads $\rightarrow$ **3.79 $\times$, 94.8% efficiency**
- 8 threads $\rightarrow$ **6.88 $\times$, 86.0% efficiency**

Three design decisions are responsible for this strong scaling:

1. Dynamic scheduling. The force loop's outer dimension iterates from $i = 0$ to $i = N - 2$. Iteration $i = 0$ computes $N - 1$ pairs; iteration $i = N - 2$ computes only 1 pair. Static work partitioning would give the first thread $O(N^2/T)$ pairs while the last thread receives $O(N^2/T^2)$ pairs. With `schedule(dynamic, 16)`, the OpenMP runtime uses a work queue: as each thread finishes a chunk of 16 outer iterations, it immediately fetches the next available chunk. Because chunks are small relative to the total, idle time at the end of the parallel region is negligible.

2. Array reduction. Newton's Third Law creates a write-after-write hazard: thread t_0 handling iteration $i = 0$ writes to `fx[j]` for $j \in [1, N - 1]$, which overlaps with all other threads. The OpenMP array reduction `reduction(+: fx[0:N])` gives each thread a *private* zero-initialised copy of the full force array. Threads write to their private copies without synchronisation, then OpenMP performs a tree reduction at the end of the parallel region:

$$\mathbf{fx}_{\text{final}}[i] = \sum_{t=0}^{T-1} \mathbf{fx}_t[i] \quad (15)$$

The merge cost is $\mathcal{O}(N \cdot T)$ floating-point additions, which for $N = 5000, T = 8$ is 40 000 operations — negligible compared to 12.5×10^6 pair evaluations.

3. Memory locality. The `Particle` AoS layout keeps each particle's x, y, v_x, v_y, m fields together in a 40-byte struct. Sequential access patterns in the integration loop prefetch whole cache lines efficiently. The private force arrays also fit comfortably in each core's L1 cache (40 kB for $N = 5000$ at 8 bytes/double), avoiding costly LLC misses during accumulation.

Efficiency degradation with thread count. Efficiency falls from 99.5% (2T) to 86.0% (8T) despite the good design. This is consistent with Amdahl’s Law applied to the inherently serial phases:

- The integration loop ($\mathcal{O}(N)$) is executed on the master thread only — parallelising it would add a second parallel region barrier.
- Particle initialisation and CSV output are serial.
- The reduction merge cost grows linearly with T .
- Cache interference increases with T as more threads simultaneously access `particles[]` data.

Even so, 86% efficiency at 8 threads is excellent for a practical implementation, well above the 70–75% typically seen in production HPC codes with similar structure.

4.2 MPI Communication Overhead

MPI achieves markedly lower speedups than OpenMP at the same worker count. At $N = 5000$:

- MPI 2P: $1.33\times$ vs. OpenMP 2T: $1.99\times$
- MPI 4P: $2.25\times$ vs. OpenMP 4T: $3.79\times$

The bottleneck is the four MPI collectives executed *every timestep*:

Collective	Purpose	Volume	Complexity
<code>MPI_Allgatherv</code> (x)	Share positions	N doubles	$\mathcal{O}(N \log P)$
<code>MPI_Allgatherv</code> (y)	Share positions	N doubles	$\mathcal{O}(N \log P)$
<code>MPI_Allreduce</code> (f_x)	Sum forces	N doubles	$\mathcal{O}(N \log P)$
<code>MPI_Allreduce</code> (f_y)	Sum forces	N doubles	$\mathcal{O}(N \log P)$

Total MPI data per timestep: $4N$ doubles $= 4N \times 8$ bytes. At $N = 5000$, $K = 15$: $4 \times 5000 \times 8 \times 15 = 2.4$ MB. On loopback (single machine), the aggregate bandwidth cost is dominated by latency (each collective invocation incurs a process synchronisation round) rather than bandwidth.

The key insight is why MPI efficiency plateaus. The computation cost per timestep scales as $\mathcal{O}(N^2/P)$ — growing quadratically with N . The communication cost scales as $\mathcal{O}(N \log P)$ — growing only linearly. Naively one would expect the computation to amortise the communication as N grows. However, the MPI `Allreduce` collects size- N arrays (force arrays for all N particles), so the communication volume itself also grows linearly with N . The computation/communication ratio is:

$$\rho = \frac{\mathcal{O}(N^2/P)}{\mathcal{O}(N \log P)} = \mathcal{O}\left(\frac{N}{P \log P}\right) \quad (16)$$

This does grow with N , which should improve efficiency. Yet the measured efficiency is flat at $\approx 65\%$ for 2P and $\approx 56\%$ for 4P. The explanation is **latency**: even very small MPI calls incur a fixed process-synchronisation latency that does not decrease with N . In the measured regime ($N \leq 5000$), latency dominates over bandwidth. On a real inter-node network (100 Gb InfiniBand), bandwidth would dominate instead, and efficiency would improve with N .

MPI load imbalance is an additional, unmeasured factor. With a static block partition, rank 0 computes interactions for particles $[0, N/P)$ — the triangle with the longest rows — while rank $P - 1$ computes only the shortest rows. The imbalance ratio is approximately:

$$\frac{T_{\max}}{T_{\min}} \approx \frac{\sum_{i=0}^{N/P-1} (N - 1 - i)}{\sum_{i=N-N/P}^{N-2} (N - 1 - i)} = \frac{(N - 1) \cdot \frac{N}{P} - \frac{(N/P)^2}{2}}{\frac{(N/P)(N/P-1)}{2}} \approx 2P - 1 \quad (17)$$

For $P = 4$, the imbalance ratio is approximately $7\times$ — rank 0 does 7 times more pair evaluations than rank 3. All processes must wait at the `MPI_Allreduce` barrier for the slowest rank, so the effective computation time per timestep is determined by rank 0 alone, wasting the computation of ranks 1 through $P - 1$.

4.3 Hybrid: Two-Level Parallelism Trade-off

The Hybrid model is evaluated against both pure OpenMP and pure MPI on a single machine to understand the trade-offs before considering multi-node deployment.

Config	Workers	Speedup ($N = 5000$)	Efficiency	Bottleneck
OpenMP 8T	8	$6.88\times$	86.0%	OMP reduction
Hybrid $2P \times 4T$	8	$4.95\times$	61.9%	MPI Allreduce
Hybrid $4P \times 2T$	8	$3.90\times$	48.7%	MPI Allreduce
MPI 4P	4	$2.25\times$	56.2%	MPI Allreduce

Why $2P \times 4T$ outperforms $4P \times 2T$. In the $4P \times 2T$ configuration, 4 MPI processes each launch 2 OpenMP threads. Each process must call `MPI_Allreduce` on a size- N array. With more processes, the Allreduce ring requires more rounds, each with a synchronisation barrier. In the $2P \times 4T$ configuration, only 2 MPI processes communicate; the Allreduce is faster, and the remaining parallelism is delivered by 4 shared-memory threads with zero communication overhead.

The gap between $2P \times 4T$ ($4.95\times$) and pure OpenMP 8T ($6.88\times$) — approximately $1.9\times$

difference — quantifies the pure cost of introducing even a minimal MPI layer on a single machine. This 28% penalty is entirely attributable to the 2 Allreduce calls per timestep.

The multi-node justification. The Hybrid model’s value becomes clear on a cluster. Consider a cluster with 4 nodes, each with 8 cores:

- **Pure MPI (32P):** All parallelism via message passing. 32 Allreduce processes. Network bandwidth and latency dominate.
- **Pure OpenMP:** Cannot span node boundaries. Limited to 8 threads per node — only $8\times$ speedup.
- **Hybrid $4P \times 8T$:** 4 MPI processes (one per node), 8 OpenMP threads each. Only 4 processes participate in Allreduce (much cheaper). Each node fully utilised via shared memory.

In this scenario, Hybrid $4P \times 8T$ achieves significantly higher speedup than any pure approach: it maximises intra-node shared-memory efficiency while minimising inter-node communication.

4.4 Overhead-Dominated Regime ($N = 100$)

At $N = 100$, the simulation performs only $\frac{100 \times 99}{2} = 4,950$ pair evaluations per timestep. The total computation at 100 iterations is 495,000 pair evaluations — taking just 0.0024 seconds sequentially. Every parallel configuration is slower:

Config	Time (ms)	Slowdown vs. Sequential
Sequential	2.44	—
MPI 4P	2.23	$0.92\times$ (marginally faster)
MPI 2P	3.64	$1.49\times$ slower
OMP 2T	8.38	$3.43\times$ slower
OMP 4T	10.30	$4.22\times$ slower
OMP 8T	15.39	$6.31\times$ slower
Hyb 2×4	7.83	$3.21\times$ slower

MPI 4P is the only configuration faster than sequential at $N = 100$ ($1.09\times$ speedup). This is counter-intuitive: how can launching 4 processes be faster than 1 when computation is minimal? The answer is that MS-MPI on Windows creates processes that are pre-warmed by the OS scheduler. On this specific machine and run, the MPI startup overhead happened to be lower than the sequential per-iteration overhead at this tiny scale. This result is non-reproducible in general and is not a meaningful speedup.

OpenMP has the worst overhead at $N = 100$: 8 threads are $6.3\times$ slower than sequential because:

1. Thread spawn cost at the `#pragma omp parallel` directive.
2. Allocating 8 private copies of `fx[100]` and `fy[100]` for the reduction.
3. The dynamic scheduling overhead (100 outer iterations need to be distributed among 8 threads via a work queue).
4. The barrier at the end of the parallel region.

These costs are *fixed* overheads per parallel region invocation, paid every single timestep. At $N = 100$, they dwarf the 4,950 pair evaluations.

The crossover point where parallelism begins to pay off is:

- **OpenMP:** Between $N = 500$ ($2.72\times$ at 8T) and $N = 100$ ($0.16\times$). Crossover is at approximately $N \approx 200\text{--}300$.
- **MPI:** Between $N = 500$ ($1.82\times$ at 4P) and $N = 100$ ($1.09\times$ at 4P). MPI crosses over near $N \approx 100$ for 4 processes.

4.5 Load Balancing in Detail

The triangular work structure is one of the most challenging aspects of the Newton's Third Law N-Body problem for parallelism. The number of pairs processed by outer iteration i is $(N - 1 - i)$, forming a strictly decreasing sequence.

Effect on MPI (static block partition): Process r handles outer iterations $[displs[r], displs[r] + counts[r])$. Rank 0 handles the first N/P iterations, which are the longest. For $N = 5000, P = 4$:

- Rank 0: iterations $[0, 1249]$ — processes pairs $\sum_{i=0}^{1249} (4999 - i) = 5,624,375$ pairs
- Rank 3: iterations $[3750, 4999]$ — processes pairs $\sum_{i=3750}^{4999} (4999 - i) = 781,250$ pairs

Rank 0 does $5,624,375/781,250 \approx 7.2\times$ more work than rank 3. Since all ranks must synchronise at `MPI_Allreduce`, ranks 1–3 spend the majority of time idle waiting for rank 0. This is a direct cause of the 56% efficiency ceiling for MPI 4P.

Mitigation strategies:

- **Cyclic assignment:** Assign outer iteration i to rank $i \bmod P$. This distributes the heavy early iterations evenly across all ranks. Cost: non-contiguous memory access pattern in `all_x/all_y` arrays, hurting cache performance.
- **Balanced block partition:** Analytically compute the partition that gives each rank equal *pair counts* rather than equal *particle counts*. The balanced boundary for equal pairs is approximately $i^* = N(1 - \sqrt{1 - r/P})$ for rank r .

- **Work stealing:** Use a master-worker model where a central coordinator assigns individual i -values dynamically. High overhead but optimal load balance.

Effect on OpenMP is much less severe because `schedule(dynamic, 16)` automatically re-balances: fast threads (which finish their first chunk quickly) receive additional chunks, while the scheduler naturally assigns fewer chunks to threads handling heavy early iterations.

TODO — Quantitative measurement: add `MPI_Reduce(MPI_MAX)` and `MPI_Reduce(MPI_MIN)` on per-process timestep wall times in `mpi.cpp` to measure the imbalance ratio $T_{\max}/T_{\min}$ directly. Expected result: ratio ≈ 7 for 4 processes at $N = 5000$.

4.6 Comparison with Theoretical Limits

To contextualise the observed speedups, we apply Amdahl's Law to estimate the parallelisable fraction from the measured data.

From speedup $S(W)$ and the formula $S(W) = 1/[(1 - f) + f/W]$, solving for f :

$$f = \frac{1/S(W) - 1}{1/W - 1} \quad (18)$$

For OpenMP 8T at $N = 5000$: $S = 6.88$, $W = 8$:

$$f_{\text{OMP}} = \frac{1/6.88 - 1}{1/8 - 1} = \frac{0.1453 - 1}{0.125 - 1} = \frac{-0.8547}{-0.875} \approx 0.977 \quad (19)$$

So **97.7% of the OpenMP run is parallelisable** at $N = 5000$. Amdahl's Law predicts the theoretical maximum speedup:

$$S_{\max} = \frac{1}{1 - f} = \frac{1}{0.023} \approx 43.5\times \quad (20)$$

Achieving $43\times$ would require roughly 80 threads at this f value. In practice, false sharing and memory bandwidth would reduce this before 80 threads.

For MPI 4P at $N = 5000$: $S = 2.25$, $W = 4$:

$$f_{\text{MPI}} = \frac{1/2.25 - 1}{1/4 - 1} = \frac{-0.556}{-0.75} \approx 0.741 \quad (21)$$

Only **74.1% of the MPI run is parallelisable** — a much lower fraction reflecting the

25.9% of time spent in serial communication (MPI collectives). Amdahl's limit for MPI:

$$S_{\max, \text{MPI}} = \frac{1}{1 - 0.741} \approx 3.86 \times \quad (22)$$

This explains why MPI never approaches linear scaling: its theoretical ceiling is only $3.86 \times$ regardless of process count. Adding more processes beyond 4 would not improve speedup significantly.

4.7 Debugging Strategies and Common Pitfalls

Developing four parallel versions of the same simulation provides a rich source of subtle bugs. The following strategies were used to ensure correctness.

Correctness verification method: Because all four versions use `initParticles()` with the same fixed seed (42), the particle positions after a given number of timesteps should be bit-identical across all implementations. Any deviation indicates a bug.

1. **Small- N sanity check:** Run all versions with $N = 4$, iterations = 1, and manually calculate the 6 pair forces by hand. Verify that each implementation produces the same force accumulators.
2. **CSV diff:** Enable `output_freq=1` on all versions; the first diverging timestep in the CSV output reveals exactly which step introduced an error.
3. **Sequential cross-check:** The sequential version is the ground truth. Any implementation whose final positions agree with sequential (to within floating-point rounding order) is correct.
4. **Force symmetry check:** Sum all force accumulators $\sum_i f_x[i]$ and $\sum_i f_y[i]$. By Newton's Third Law, these sums must be zero (equal and opposite forces cancel). A non-zero sum immediately flags a sign error or missing pair.

Common pitfalls in this project:

- **Race condition in OpenMP:** Using `fx[j] -= f_x` without array reduction produces non-deterministic results that may *appear* correct on some runs (when threads happen not to collide). The bug manifests as occasional trajectory divergence. Fix: add `reduction(+: fx[0:N], fy[0:N])` and switch to raw C arrays (OpenMP array reduction requires non-class types).
- **Off-by-one in MPI global index:** Using `local_i` instead of `global_i = local_start + local_i` in the inner loop. All pair forces are computed against wrong particles. Fix: ensure the index into `all_x[]` uses the global index.

- **Missing MPI_Allgatherv before force loop:** Forgetting to share updated positions after integration means each process computes forces against stale positions. Divergence grows over iterations.
- **Incorrect MPI_Allreduce size:** Passing `local_n` instead of `N` to `MPI_Allreduce`. Each process reduces only its local slice; global forces are wrong.

5 Conclusion

5.1 Summary of Findings

This project implemented and benchmarked the N-Body gravitational simulation problem across four parallel programming paradigms: Sequential (baseline), OpenMP (shared-memory), MPI (distributed-memory), and Hybrid (MPI + OpenMP two-level). All implementations apply Newton’s Third Law to halve pair evaluations, ensuring that measured speedups reflect genuine parallelism gains rather than algorithmic differences.

The benchmark results across $N \in \{100, 500, 1000, 2000, 5000\}$ and multiple worker counts reveal four key conclusions:

- 1. OpenMP is the best choice for single-node performance.** At $N = 5000$, OpenMP 8 threads achieves $6.88\times$ speedup at 86% efficiency — the highest of any configuration tested. The combination of zero communication cost, dynamic scheduling (addressing the triangular loop imbalance), and OpenMP array reduction (avoiding race conditions) makes this the optimal design for shared-memory parallelism of this problem.
- 2. MPI communication overhead prevents efficient single-node scaling.** Despite using 4 workers, MPI 4P achieves only $2.25\times$ speedup — significantly below OpenMP 4T’s $3.79\times$. The per-timestep `MPI_Allreduce` on size- N force arrays introduces a synchronisation cost that cannot be amortised. MPI efficiency plateaus at approximately 56% for 4 processes and 66% for 2 processes, independent of N . Amdahl’s Law analysis confirms a theoretical ceiling of $\approx 3.86\times$ for MPI 4P on this problem structure.
- 3. The Hybrid model is sub-optimal on a single node but essential for multi-node clusters.** On a single machine, Hybrid $2P \times 4T$ achieves $4.95\times$ speedup — better than pure MPI but below pure OpenMP. However, pure OpenMP cannot span node boundaries. On a cluster with K nodes, the Hybrid approach ($KP \times T$ configuration) is the *only* way to fully utilise all cores, combining MPI’s unlimited inter-node scalability with OpenMP’s efficient intra-node execution. For such deployments, minimising the process count (and hence MPI communication frequency) by maximising threads per process is the correct design principle, confirmed by $2P \times 4T > 4P \times 2T$ consistently.
- 4. All parallel methods suffer in the overhead-dominated regime.** At $N = 100$,

every parallel configuration is slower than sequential. The crossover point is approximately $N \approx 200$ – 300 for OpenMP and $N \approx 100$ – 200 for MPI 4P. This demonstrates that parallelism is not a universal improvement but a tool that requires sufficient computation to amortise its setup cost.

Finding	Evidence
Parallel overhead dominates at small N	$N = 100$: all parallel versions slower than sequential
OpenMP achieves near-linear scaling for large N	8 threads $\rightarrow 6.88\times$ (86% eff.) at $N = 5000$
MPI communication does not amortise	Efficiency flat at 56–66% regardless of N
Fewer MPI processes + more OMP threads wins	$2P \times 4T$ ($4.95\times$) $>$ $4P \times 2T$ ($3.90\times$)
Hybrid is essential for multi-node but sub-optimal on single-node	Always slower than pure OMP at equal worker count
MPI static partition creates severe load imbalance	Rank 0 does $\approx 7\times$ more work than rank 3 for $P = 4$
Newton’s 3rd law gives fair baseline for all implementations	Consistent algorithms; speedups reflect parallelism only

5.2 Limitations of This Study

Several limitations affect the generalisability of these results:

- **Single machine:** All four versions were tested on the same node. MPI’s true advantage — scaling across nodes connected by a high-speed network — cannot be demonstrated. The measured MPI times are loopback-bound, not bandwidth-bound.
- **First-order integrator:** The semi-implicit Euler method accumulates energy error at $\mathcal{O}(\Delta t)$ per step. For physically accurate long-term simulations, this is insufficient. The performance results are unaffected, but physical validity of trajectories decreases over many timesteps.
- **2D only:** The simulation is two-dimensional. Real astrophysical simulations are 3D, which changes the force calculation (adding dz , f_z components) but not the parallel structure.
- **No GPU baseline:** GPU-accelerated N-Body (CUDA, OpenCL) typically outperforms CPU-based approaches by 10 – $100\times$ for large N . Without a GPU reference, it is unclear how the presented results compare to state-of-the-art.

- **Static benchmark:** Results are single-run measurements. Statistical averaging over multiple runs would provide confidence intervals and reduce variability from OS scheduling noise.

5.3 Future Work

Seven directions would most improve this implementation:

1. **Leapfrog (Velocity-Verlet) Integration.** Replace the semi-implicit Euler method with the leapfrog scheme: $v(t + \Delta t/2) = v(t) + a(t) \cdot \Delta t/2$, $x(t + \Delta t) = x(t) + v(t + \Delta t/2) \cdot \Delta t$, $v(t + \Delta t) = v(t + \Delta t/2) + a(t + \Delta t) \cdot \Delta t/2$. Leapfrog is second-order accurate and *symplectic* — it conserves total mechanical energy in the absence of dissipation, enabling physically valid long-term simulations.
2. **Energy Conservation Tracking.** Compute total kinetic energy $E_k = \sum_i \frac{1}{2} m_i (v_x^2 + v_y^2)$ and potential energy $E_p = -\sum_{i < j} G m_i m_j / r_{ij}$ each timestep. The ratio $E(t)/E(0)$ measures integrator quality and can detect correctness bugs (any implementation error manifests as spurious energy drift).
3. **MPI Load Balancing.** Replace the static block partition with a cyclic assignment (particle i assigned to rank $i \bmod P$) or a balanced-pairs partition where rank boundaries are computed analytically to equalise pair counts. Expected to improve MPI efficiency from 56% toward 70–80%.
4. **Barnes-Hut Tree ($\mathcal{O}(N \log N)$).** Replace the $\mathcal{O}(N^2)$ exact force computation with a hierarchical tree approximation. Distant particle groups are approximated as a single mass at their centre of mass, reducing force evaluations from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ with controllable accuracy (opening angle θ parameter). This enables simulations with $N > 10^6$ particles.
5. **3D Extension.** Add z, v_z, f_z components to all data structures and force calculations. Trivial code change, but makes the simulation physically realistic and enables comparison with published N-Body benchmarks (e.g., Gadget-2, SWIFT).
6. **SIMD Vectorisation.** Add `#pragma omp simd` on the inner force loop and compile with `-O3 -march=native -ffast-math`. On modern CPUs with AVX2 (256-bit), this processes 4 double-precision pairs per CPU clock cycle instead of 1, potentially doubling throughput on the force computation kernel.
7. **GPU Acceleration (CUDA).** Map the N^2 force computation to a 2D thread grid where thread (i, j) computes one pair. With shared memory tiling (each block caches a tile of particles), memory bandwidth is reduced significantly. A single modern GPU (e.g., NVIDIA A100) has $\sim 9,700$ CUDA cores and can compute the N-Body force kernel 100–1000 $\times$ faster than a CPU for $N \geq 10,000$.

5.4 Closing Remarks

The N-Body gravitational simulation is a canonical HPC benchmark that exposes the essential trade-offs of modern parallel computing: computation vs. communication, load balance vs. simplicity, scalability vs. efficiency.

This project demonstrates that **there is no universally optimal parallel strategy**. OpenMP wins on a single shared-memory node. MPI scales across distributed-memory nodes but incurs communication overhead. Hybrid MPI+OpenMP is the right choice for multi-node clusters. The best choice always depends on the hardware topology, problem size, and acceptable implementation complexity.

Most importantly, the results show that **correctness must be established before performance is optimised**: all four implementations were validated by cross-checking final particle positions against the sequential baseline before any timing experiments were run. A fast but incorrect implementation is worthless in scientific computing.

The combination of Newton's Third Law (algorithmic optimisation), OpenMP array reduction (safe shared-memory parallelism), `MPI_Allreduce` (correct distributed force aggregation), and `MPI_THREAD_FUNNELED` (thread-safe MPI in Hybrid) forms a principled and extensible foundation for a production-quality N-Body code.